Minimum Cost to Connect Ropes

1. Problem Title & Link

Title: **Minimum Cost to Connect Ropes**

Link: — (Classic Greedy / Min Heap problem; similar pattern used in Huffman Coding and LeetCode 1584 style greedy optimization)

2. Problem Statement (Clear Explanation)

You are given a list of ropes where each rope has a certain **length**.

You want to connect all ropes into **one single rope**.

When you connect two ropes of lengths a and b:

cost = a + b

After connecting them:

new rope length = a + b

This newly formed rope can again be used to connect with other ropes.

Your goal is to determine the **minimum total cost required to connect all ropes into one rope**.

Key idea:

To minimize cost, you should **always connect the two smallest ropes first**.

3. Examples (Input → Output)

Example 1

Input

ropes = [4,3,2,6]

Output

29

Explanation

Step-by-step:

2 + 3 = 5 cost = 5

ropes → [4,5,6]

4 + 5 = 9 cost = 9

ropes → [6,9]

6 + 9 = 15 cost = 15

Total cost:

5 + 9 + 15 = 29

Example 2

Input

ropes = [1,2,3]

Output

9

Explanation

1 + 2 = 3 cost = 3

ropes → [3,3]

3 + 3 = 6 cost = 6

Total cost:

3 + 6 = 9

Example 3

Input

ropes = [5]

Output

0

Explanation

Only one rope exists, so no connections are needed.

4. Constraints

1 ≤ ropes.length ≤ 100000

1 ≤ ropes[i] ≤ 100000

Additional considerations:

- Ropes may have **duplicate lengths**
- Large inputs require **efficient algorithm**
- Expected complexity close to **O(n log n)**

5. Core Concept (Pattern / Topic)

Greedy

Min Heap / Priority Queue

6. Thought Process (Step-by-Step Explanation)

Step 1 — Brute Force Idea

We could try **all possible combinations** of rope connections.

However:

Number of combinations grows exponentially

This approach becomes infeasible.

Step 2 — Greedy Observation

Connecting **large ropes early increases future costs**.

Example:

ropes = [2,3,6,7]

Bad choice:

6 + 7 = 13

This large rope will increase later connection costs.

Better strategy:

2 + 3 = 5

Small ropes produce **smaller intermediate ropes**, keeping total cost low.

Step 3 — Greedy Strategy

Always connect:

two smallest ropes

To efficiently get the smallest ropes repeatedly, use a:

Min Heap (Priority Queue)

Step 4 — Algorithm

1. Insert all ropes into a **min heap**
2. While heap size > 1:
    - Extract two smallest ropes
    - Connect them
    - Add cost
    - Insert new rope back into heap

7. Visual / Intuition Diagram

Example:

ropes = [4,3,2,6]

Heap:

[2,3,4,6]

Steps:

2 + 3 = 5

Heap:

[4,5,6]

4 + 5 = 9

Heap:
[6,9]

6 + 9 = 15
Total cost:
29

8. Pseudocode (Language Independent)

```
function minCost(ropes):

   create minHeap from ropes

   total_cost = 0

   while heap size > 1:

      a = extractMin()
      b = extractMin()

      cost = a + b

      total_cost += cost

      insert cost into heap

   return total_cost
```

9. Code Implementation

Python

```
import heapq

def minCost(ropes):

   heapq.heapify(ropes)
```

```
    total_cost = 0

    while len(ropes) > 1:

        a = heapq.heappop(ropes)
        b = heapq.heappop(ropes)

        cost = a + b

        total_cost += cost

        heapq.heappush(ropes, cost)

    return total_cost
```

Java

```
import java.util.*;

class Solution {

    public int minCost(int[] ropes) {

        PriorityQueue<Integer> pq = new PriorityQueue<>();

        for(int r : ropes)
            pq.add(r);

        int totalCost = 0;

        while(pq.size() > 1) {

            int a = pq.poll();
            int b = pq.poll();

            int cost = a + b;

            totalCost += cost;
```

```
            pq.add(cost);
        }

        return totalCost;
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n log n)

Reason:

- Heap operations take log n
- Performed n-1 times

Space Complexity

O(n)

Heap stores all ropes.

11. Common Mistakes / Edge Cases

Common mistakes:

1. Connecting ropes in **random order**
2. Sorting once and not maintaining order
3. Forgetting to push new rope back to heap

Edge cases:

ropes = [5]

ropes = [1,1,1]

ropes = [100000]

12. Detailed Dry Run

Example:

ropes = [4,3,2,6]

Initial heap:

[2,3,4,6]

Step	Ropes	Cost	Total
1	2+3=5	5	5
2	4+5=9	9	14

3	6+9=15	15	29

Result:

29

13. Common Use Cases (Real-Life / Interview)

This greedy pattern appears in:

- **File merging optimization**
- **Huffman coding (data compression)**
- **Network cable optimization**
- **Task merging cost problems**

14. Common Traps (Important!)

1. Sorting once and not updating order
2. Using max heap instead of min heap
3. Forgetting to reinsert merged rope

15. Builds To (Related LeetCode Problems)

Progression:

LC 1046 Last Stone Weight

LC 1167 Minimum Cost to Connect Sticks

LC 1584 Min Cost to Connect All Points

LC 1135 Connecting Cities With Minimum Cost

16. Alternate Approaches + Comparison

Approach	Time	Notes
Brute Force	Exponential	Impossible
Sorting repeatedly	$O(n^2 \log n)$	Inefficient
Min Heap	$O(n \log n)$	Optimal

17. Why This Solution Works (Short Intuition)

Connecting the **smallest ropes first keeps intermediate rope lengths minimal**, preventing large costs from propagating through later merges.

18. Variations / Follow-Up Questions

Possible interview extensions:

1. Ropes have **connection limits**
2. Each connection has **additional fixed cost**
3. Return **actual connection sequence**
4. Multiple ropes can be connected **at once**.
5.